

UTILITY-PEFT: LEARNING THE COUNTERFACTUAL VALUE OF ADAPTATION MODULES FOR TIME-SERIES FOUNDATION MODELS

Renan Julia Martins Frejat Chi-Nan Pai Artur Jordão
University of São Paulo / Escola Politécnica *working author list placeholder*

ICLR-style research and implementation proposal for PCS 5022 / PhD article planning

Abstract

Time-series foundation models (TSFMs) can transfer across datasets and tasks, but the best parameter-efficient fine-tuning (PEFT) procedure is strongly target-dependent. Recent time-series-specific methods expose an important limitation of importing low-rank allocation heuristics from natural language processing: global, static, or irreversible capacity allocation can suppress locally useful temporal behavior. Time-PEFT addresses temporal and multichannel complexity with fixed frequency and channel adapters; TRACE selects LoRA modules through target-specific masking-based importance estimation; TS-PET allocates ranks using interaction-aware pruning; AT4TS performs target-specific fine-tuning search; and MixFT trains a library of adapters for inferred temporal subdomains. These methods improve adaptation, but none directly predicts the *counterfactual downstream value* of activating a candidate adaptation operation on a previously unseen dataset or regime.

We propose Utility-PEFT, an amortized, budget-aware policy that estimates the expected query-set improvement of candidate adapter families, LoRA layer groups, and effective ranks before full adaptation. Given a small support episode, the policy combines time-series structure, frozen-model mismatch, and cheap gradient probes to predict the utility and interaction of candidate modules. It then selects a sparse configuration maximizing expected improvement minus parameter, memory, and compute costs. The primary implementation uses MOMENT-base, a finite adapter action space, support/query episodic training, and a learning-to-rank utility predictor. Evaluation spans the forecasting datasets used by Time-PEFT, held-out multivariate classification tasks, and EMG adaptation through NinaPro DB6, NinaPro DB3, and EMGBench. The proposal includes an implementation blueprint, controlled baselines, resource-matched protocols, ablations, falsification criteria, and a staged plan suitable for implementation by an autonomous coding agent in a GPU environment.

1 Introduction

Parameter-efficient fine-tuning adapts a frozen foundation model by training a small set of additional or selected parameters. LoRA represents a weight update as a low-rank product and is widely used because it reduces trainable parameters while preserving the pretrained backbone [Hu et al., 2022]. However, uniform ranks and manually selected insertion points can be suboptimal. AdaLoRA allocates rank through importance scores derived from singular triplets [Zhang et al., 2023]; AutoLoRA uses meta-learning to select rank-one components [Zhang et al., 2024]; FlexLoRA permits rank pruning and expansion under a global budget [Liu et al., 2026]; and CoTo uses stochastic adapter activation and cooperative-game analysis of marginal contribution [Zhuang et al., 2025]. These methods show that adaptation capacity is neither uniform nor equally useful across model components.

The mismatch is particularly acute for time series. Temporal patterns may be localized, nonstationary, multiscale, and distributed unevenly across channels. TRACE argues that global singular-value importance and irreversible early pruning can overlook local temporal events and later-useful components; it instead uses reversible gates and simulated masking to estimate target-specific LoRA module importance [Li and Zhu, 2026]. Time-PEFT introduces dataset-level temporal complexity based on spectral entropy and multichannel complexity based on lagged transfer entropy, then augments LoRA

with a frequency adapter and a channel adapter [Na et al., 2026]. TS-PET proposes interaction-aware rank allocation and a lightweight prediction head [Zheng and Jin, 2026]. AT4TS automates selective fine-tuning through target-specific hyperparameter optimization [Tomar et al., 2025], while MixFT discovers latent temporal subdomains and trains one LoRA module per inferred component [Lee et al., 2026].

These papers establish that *time-series adaptation must be specialized*. They leave a sharper question open:

Can a model predict, from a small target episode, which adaptation operation will actually improve downstream performance and justify its computational cost, without running a new exhaustive search or target-specific pruning procedure?

This question is different from estimating data complexity. Spectral entropy and transfer entropy describe the input distribution, but they do not directly measure whether the frozen model already represents that structure or whether activating a specific adapter will improve the target task. Similarly, gradient importance measured after repeated target-specific training does not provide an amortized decision rule transferable across datasets. We therefore formulate adaptation selection as *counterfactual utility prediction*.

For an episode e and adaptation action a , the utility is the normalized query-loss reduction obtained after adapting on the support set, minus a weighted resource cost. During meta-training, actions are evaluated on source datasets to construct utility labels. A learned policy then predicts utilities for unseen episodes using three evidence families: (i) time-series structure; (ii) mismatch between the frozen model and the target episode; and (iii) cheap gradient probes estimating the responsiveness of candidate modules. At deployment, the policy chooses the best action under a budget and only then performs adaptation.

Research goal. Develop and validate a transferable adaptation-value predictor that selects PEFT components for unseen time-series datasets, tasks, subjects, and sessions more accurately and cheaply than fixed configurations, complexity thresholds, target-specific search, or pruning-based allocation.

Main contributions proposed.

1. A distinction between *data complexity* and *adaptation utility*, formalizing the latter as the counterfactual value of a PEFT action under an explicit resource budget.
2. A modular TSFM adaptation bank spanning the task head, LoRA layer groups and ranks, frequency adaptation, and channel adaptation.
3. An amortized utility predictor using structural descriptors, frozen-model mismatch, and one-step gradient probes, with pairwise terms for adapter synergy and conflict.
4. A cross-dataset episodic training and evaluation protocol that measures oracle regret, negative transfer, configuration-ranking quality, and adaptation cost.
5. A staged implementation and experiment plan covering forecasting, generic multivariate classification, and longitudinal/cross-subject EMG adaptation.

2 Novelty boundary and related work

2.1 Time-series-specific PEFT

Time-PEFT measures temporal complexity through normalized spectral entropy and multichannel complexity through lagged transfer entropy. These statistics serve as indicators of where its specialized frequency and channel adapters provide larger gains [Na et al., 2026]. The architecture, however,

uses a fixed combination of LoRA, frequency, channel, and forecast-head modules with predefined LoRA layers and ranks. Its own results show that additional modules can be unnecessary on lower-complexity standard datasets. This motivates selection, but a rule such as “high transfer entropy implies channel adapter” would be only an incremental extension.

TRACE replaces AdaLoRA-style importance with Gated Dynamic Simulation Importance Calculation and reconstructs large forecasting heads [Li and Zhu, 2026]. Its selection is reversible and target-specific, but requires importance estimation during adaptation on each target. TS-PET estimates synergistic rank interactions and uses stochastic approximation, but it remains a target-specific rank-allocation method [Zheng and Jin, 2026]. MixFT routes contexts to adapters trained for discovered subdomains, whereas Utility-PEFT aims to compose an adaptation configuration for an unseen target rather than retrieve a previously trained subdomain expert [Lee et al., 2026]. AT4TS is an important search baseline because it automates selective fine-tuning, but its optimization cost is paid again for each target domain [Tomar et al., 2025].

2.2 Automatic and meta-learned PEFT

AdaLoRA, AutoLoRA, dEBORA, FlexLoRA, and LoRA-Drop address rank or module allocation using importance, meta-gradients, bilevel optimization, entropy, or pruning [Zhang et al., 2023, 2024, Zangrando et al., 2025, Liu et al., 2026, Zhou et al., 2025]. Meta-Adapters and recent provable PEFT meta-learning prepare models for rapid low-dimensional adaptation across tasks [Bansal et al., 2022, Block et al., 2025]. These works motivate amortization across tasks, but do not predict the utility of heterogeneous time-series adapter families under a resource budget.

CoTo is particularly relevant because it quantifies adapter marginal contributions using a cooperative-game view [Zhuang et al., 2025]. Utility-PEFT differs in objective and deployment: CoTo improves training and post-hoc operations for a given target, whereas Utility-PEFT predicts target-episode-specific counterfactual gains before committing to full adaptation. The comparison nevertheless requires care, and a CoTo-style marginal-contribution estimator is included as a baseline or label-acceleration mechanism.

2.3 Time-series foundation models and tasks

MOMENT is the primary backbone because it is open, supports multiple downstream tasks, and is used by Time-PEFT and related TSFM adaptation work [Goswami et al., 2024]. UniTS is the secondary backbone because it unifies forecasting, classification, anomaly detection, and imputation across heterogeneous domains [Gao et al., 2024a]. Chronos, TimesFM, Tiny Time Mixers, Sundial, and other TSFMs provide additional context, but the first article should avoid an unnecessarily broad backbone matrix [Ansari et al., 2024, Das et al., 2024, Ekambaram et al., 2024, Liu et al., 2025].

2.4 EMG as a held-out adaptation domain

EMGBench standardizes intersubject generalization and adaptation across nine electromyography datasets [Yang et al., 2024]. NinaPro DB6 contains repeated grasp sessions across five days and is therefore suitable for chronological session/day shifts [Palermo et al., 2017]. NinaPro DB3 contains transradial amputee participants and provides a clinically meaningful population shift [Atzori et al., 2015]. EMG is not the source of the method’s novelty; it is a demanding held-out domain where temporal structure, channel relations, sensor placement, session drift, and user variation can make adaptation choices materially different.

3 Problem formulation

3.1 Episodes, backbone, and adaptation actions

Let f_θ be a pretrained TSFM with frozen parameters θ . An adaptation episode is

$$e = (\mathcal{D}_e^{\text{sup}}, \mathcal{D}_e^{\text{qry}}), \quad (1)$$

where the support set is available for fine-tuning and the query set is used only to measure the consequence of the adaptation action during meta-training or evaluation. The target deployment setting may be supervised few-shot, weakly supervised, or source-free; the primary implementation uses labeled few-shot support to isolate the adaptation-selection problem cleanly.

An action $a \in \mathcal{A}$ specifies a subset and configuration of adaptation modules. The minimum viable action space is

$$\mathcal{A}_{\text{MVP}} = \{\emptyset, H, H+L, H+L+F, H+L+C, H+L+F+C, H+\text{FourierFT}\}, \quad (2)$$

where H is the task head, L is fixed-rank LoRA on predefined backbone projections, F is a frequency adapter, and C is a channel adapter. Full fine-tuning is an upper reference rather than a deployable controller action.

The extended action space factorizes an action as

$$a = (S_a, G_a, r_a, b_a), \quad (3)$$

where S_a is the adapter-family subset, G_a is the selected layer group, r_a is the effective LoRA rank, and b_a is the update budget such as epochs or steps.

3.2 Counterfactual adaptation utility

Let $\ell_0(e)$ denote the query loss of the frozen or head-only reference, and let $\ell_a(e)$ denote the query loss after adapting action a on $\mathcal{D}_e^{\text{sup}}$. Define normalized performance gain

$$\Delta(e, a) = \frac{\ell_0(e) - \ell_a(e)}{|\ell_0(e)| + \varepsilon}. \quad (4)$$

Let c_p , c_f , c_m , and c_t denote normalized active-parameter, backward-FLOP, peak-memory, and wall-clock costs. The action utility is

$$U(e, a) = \Delta(e, a) - \lambda_p c_p(a) - \lambda_f c_f(a) - \lambda_m c_m(a) - \lambda_t c_t(a). \quad (5)$$

The nondeployable oracle action is

$$a_e^* = \arg \max_{a \in \mathcal{A}} U(e, a). \quad (6)$$

The goal is to learn $\hat{U}_\psi(e, a)$ such that

$$\hat{a}_e = \arg \max_{a \in \mathcal{A}: \mathcal{C}(a) \leq B} \hat{U}_\psi(e, a) \quad (7)$$

achieves low oracle regret on datasets and regimes excluded from policy training.

3.3 Why utility is different from complexity

Time-series complexity is a property of the episode. Adaptation utility is a property of the tuple (e, a, f_θ) . Two episodes can have similar spectral and transfer entropy but different utility because the frozen model represents one well and the other poorly. Likewise, a highly multichannel dataset need not benefit from a channel adapter if the backbone already models the relevant dependence. This motivates conditioning on the frozen model and candidate action rather than routing from raw complexity thresholds alone.

4 Proposed method: Utility-PEFT

4.1 System overview

4.2 Modular adapter bank

The first implementation uses a shared modular bank:

$$\mathcal{M} = \{H, L, F, C\}. \quad (8)$$

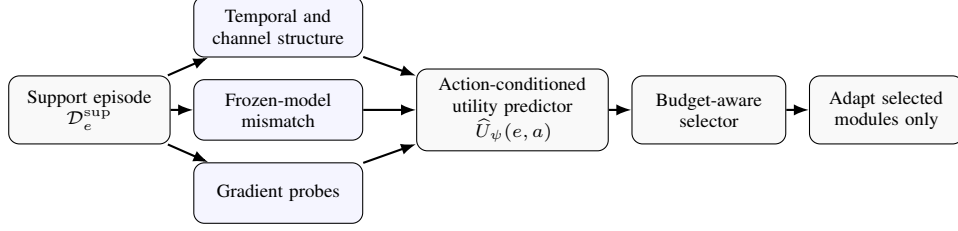


Figure 1: Utility-PEFT predicts the value of candidate adaptation operations before full fine-tuning. Transfer entropy is an optional structural feature, not the routing rule or source of novelty.

Head H . A task-specific linear or shallow prediction head. It is always available and may be trained alone.

LoRA L . For a selected weight matrix $\mathbf{W}_\ell$,

$$\mathbf{W}'_\ell = \mathbf{W}_\ell + \alpha_\ell \sum_{j=1}^{R_{\max}} g_{\ell j} b_{\ell j} a_{\ell j}^\top. \quad (9)$$

The MVP fixes the target projections and $r = 8$. The extended implementation uses nested rank-one units, enabling effective ranks $r \in \{2, 4, 8, 16\}$ without SVD pruning.

Frequency adapter F . A Time-PEFT-compatible module that transforms backbone embeddings along the patch/time axis, retains dominant or learned frequency components, reconstructs a filtered representation, and combines it with the original representation [Na et al., 2026]. The first reproduction should match Time-PEFT’s top- k implementation before introducing learned or localized alternatives.

Channel adapter C . A shared down-projection and channel-specific up-projection module inspired by Time-PEFT, adapted to both forecasting and classification heads [Na et al., 2026]. For very high channel counts, grouped or low-rank channel projections are required to control parameter growth.

4.3 Evidence available to the utility predictor

The episode representation is

$$\mathbf{z}_e = [\mathbf{z}_e^{\text{struct}} \parallel \mathbf{z}_e^{\text{mismatch}} \parallel \mathbf{z}_e^{\text{grad}} \parallel \mathbf{z}_e^{\text{meta}}]. \quad (10)$$

Time-series structure. Candidate descriptors include normalized spectral entropy, spectral concentration, autocorrelation decay, trend strength, multiscale energy, transient/change-point density, channel count, sampling metadata, transfer-entropy summaries, and cheap alternatives such as lagged correlation or mutual information. Transfer entropy should be treated as an ablated feature because its $O(C^2L)$ cost can become prohibitive for dense EMG.

Frozen-model mismatch. Candidate signals include frozen support loss, representation distance to source prototypes, masked-reconstruction loss, prediction entropy, calibration, feature covariance shift, and disagreement under time-series augmentations. These features distinguish data complexity from whether the pretrained model already handles it.

Gradient probes. Initialize all candidate modules with zero-impact parameterization, perform one support backward pass, and record group-level statistics without applying an update. For module m :

$$q_m(e) = [\|g_m\|_2, \|g_m\|_\infty, \text{Var}(g_m), \text{SNR}(g_m), \cos(g_m, g_H)], \quad (11)$$

where $g_m = \nabla_{\phi_m} \mathcal{L}_{\text{sup}}$. The first-order predicted gain $\eta \|g_m\|_2^2$ is included as a feature, not assumed to be an exact utility estimate.

Episode metadata. Support size, number of channels, lookback length, horizon, task type, and backbone identity are encoded explicitly. Dataset names are never provided to the controller in held-out-dataset evaluation.

4.4 Action-conditioned utility predictor

Each action has an embedding v_a describing its active module families, layer groups, rank, and budget. The predictor receives $(\mathbf{z}_e, v_a)$ and outputs a mean and uncertainty:

$$(\mu_{ea}, \log \sigma_{ea}^2) = h_\psi(\mathbf{z}_e \| v_a). \quad (12)$$

A heteroscedastic regression loss is combined with pairwise ranking:

$$\mathcal{L}_{\text{reg}} = \sum_{e,a} \frac{(U(e,a) - \mu_{ea})^2}{2\sigma_{ea}^2} + \frac{1}{2} \log \sigma_{ea}^2, \quad (13)$$

$$\mathcal{L}_{\text{rank}} = \sum_{e,a^+,a^-} \max\{0, \gamma - \mu_{ea^+} + \mu_{ea^-}\}, \quad (14)$$

$$\mathcal{L}_{\text{utility}} = \mathcal{L}_{\text{reg}} + \beta \mathcal{L}_{\text{rank}}. \quad (15)$$

Ranking is more important than exact calibration because deployment requires choosing a good action. Utility uncertainty supports conservative selection: if the expected gain is smaller than its uncertainty-adjusted margin, the policy can choose the no-op or head-only action.

4.5 Interaction-aware utility

Adapter effects need not be additive. Time-PEFT reports synergy between LoRA, frequency, and channel modules, while TS-PET and CoTo motivate interaction-aware importance [Na et al., 2026, Zheng and Jin, 2026, Zhuang et al., 2025]. A factorized alternative predicts

$$\hat{U}(e, S) = \sum_{m \in S} \hat{u}_m(e) + \sum_{m < n; m, n \in S} \hat{u}_{mn}(e) - \lambda \mathcal{C}(S). \quad (16)$$

The MVP can directly enumerate seven actions. The factorized formulation becomes useful when layer groups and ranks expand the action space.

4.6 Budget-aware selection

For the MVP, enumerate all actions satisfying the active-parameter or backward-FLOP budget. For the extended action space, use beam search or a small binary integer program. The selector should support three deployment policies:

1. **Accuracy-first:** maximize predicted gain with a soft cost penalty.
2. **Hard budget:** maximize predicted gain subject to parameter, memory, or latency constraints.
3. **Conservative:** adapt only when a lower confidence bound on utility is positive.

4.7 Constructing counterfactual utility labels

Exact utility labels require adapting each candidate action, which is expensive but feasible for a small action set and moderate backbone. Labels are generated offline and cached. To scale beyond the MVP:

1. use low-fidelity actions with fewer steps or smaller support subsets;
2. apply successive halving to remove clearly inferior actions;
3. exploit nested LoRA ranks so one maximum-rank run can yield partial-rank evaluations;
4. warm-start compatible action supersets;
5. use CoTo/TRACE-style masking estimates as auxiliary targets, never as the sole ground truth;
6. retain exact full-budget labels for a stratified subset to calibrate low-fidelity bias.

4.8 Deployment settings

Primary: labeled few-shot adaptation. A small labeled support set is available, as in EMGBench adaptation protocols. This is the cleanest setting for validating utility selection.

Secondary: unlabeled/source-free adaptation. Replace supervised support loss with masked reconstruction, consistency, entropy, or pseudo-label losses. The controller is either retrained for this objective or receives the objective type as part of the action. This extension should follow after the primary claim is established.

Online session adaptation. For NinaPro DB6, recompute episode evidence at each new session and decide whether to preserve, reset, or replace the adapter state. This is a deployment extension, not required for the first complete result.

5 Implementation strategy

5.1 Staged scope

5.2 Recommended repository layout

```
utility-peft/  
  configs/  
    backbone/          # moment_base.yaml, units.yaml  
    dataset/           # forecasting and EMG protocols  
    action_space/      # mvp.yaml, extended.yaml  
    experiment/        # reproduction, oracle_map, meta_train, heldout_eval  
  src/utility_peft/  
    backbones/         # unified encode/predict interface  
    adapters/          # head, LoRA bank, frequency, channel, FourierFT  
    episodes/          # support/query sampling and leakage checks  
    descriptors/       # structural and mismatch features  
    probes/            # gradient-hook statistics  
    utility/           # action evaluator, cost model, cache schema  
    controller/        # action encoder, utility predictor, selector  
    training/          # adapter, label-generation, meta-training loops  
    metrics/           # task, cost, regret, ranking, calibration  
  scripts/  
    reproduce_time_peft.py  
    generate_utility_table.py  
    train_utility_predictor.py  
    evaluate_heldout.py  
  tests/  
    test_no_target_leakage.py  
    test_zero_impact_adapters.py
```

Table 1: Implementation stages. Each stage should produce a testable artifact before expanding the action space.

Stage	Scope	Completion criterion
0	Reproduce MOMENT, LoRA, FourierFT, and Time-PEFT on a small subset	Results within reported variability or documented implementation discrepancy
1	Build episode API, adapter bank, exact action evaluator, and cache	Deterministic utility table for at least three datasets
2	Train utility predictor on structural, mismatch, and gradient evidence	Utility ranking exceeds complexity-only and random baselines
3	Leave-one-dataset-out forecasting evaluation	Low oracle regret and favorable accuracy-cost frontier on unseen datasets
4	Extend to MOMENT classification and EMGBench/NinaPro	Policy transfers across task and domain boundaries
5	Add rank/layer factorization and secondary UniTS backbone	Gains survive larger action space and architecture change
6	Optional source-free/online TTT extension	Stable adaptation without target-label leakage

```
test_action_budget.py
test_episode_chronology.py
Makefile
pyproject.toml
README.md
```

5.3 Core software interfaces

Backbone adapter. Every backbone wrapper should expose:

```
encode(x, mask=None) -> embeddings
predict(x, task_spec) -> predictions
adapter_targets() -> named parameter groups
source_statistics() -> optional representation prototypes
```

Action specification.

```
ActionSpec(
    modules={"head", "lora", "frequency", "channel"},
    layer_group="last_half",
    rank=8,
    update_steps=100,
    budget={"max_trainable_pct": 4.0}
)
```

Episode specification.

```
Episode(
    support_x, support_y,
    query_x, query_y,
    task_type, dataset_family,
    subject_id=None, session_id=None,
    chronology_metadata=None
)
```

Utility record.

```
UtilityRecord(
    episode_id, action_id, seed,
    frozen_loss, adapted_loss,
    trainable_params, backward_flops,
    peak_memory_mb, wall_time_s,
    utility, descriptors, gradient_probes
)
```

5.4 Reproducibility and engineering requirements

1. Pin exact model checkpoints, dataset versions, package commits, and preprocessing hashes.
2. Store every split manifest and verify that target query labels never influence action selection.
3. Use deterministic episode IDs and cache all action evaluations in Parquet or SQLite.
4. Separate *stored parameters* from *active trainable parameters*; report both.
5. Log measured GPU memory and wall-clock time rather than relying only on analytical counts.
6. Add smoke tests with a tiny backbone and synthetic time series before GPU-scale runs.
7. Implement resume-safe utility-table generation because this is the most expensive phase.
8. Use Hydra or an equivalent configuration system so every table row can be regenerated from a committed config.

6 Experimental design**6.1 Backbones**

Primary backbone: MOMENT-base. MOMENT provides open checkpoints and supports forecasting and classification [Goswami et al., 2024]. It also yields the cleanest comparison with Time-PEFT and TRACE.

Secondary backbone: UniTS. UniTS is used only after the method is stable on MOMENT, testing whether the controller overfits one architecture [Gao et al., 2024a].

6.2 Datasets

Forecasting reproduction and meta-training pool. Use the Time-PEFT suite:

- complex/chaotic: Lorenz, CellCycle, DoublePendulum, Hopfield, LorenzCoupled;
- medical: ECGCA115 and ECGCA15;
- standard: ETTh1, ETTh2, ETTm1, ETTm2, Weather, Exchange.

The initial protocol follows lookback 96 and horizons {96, 192, 336} for comparability, with model-specific exceptions documented [Na et al., 2026].

Generic multivariate classification. Select a stratified subset of the UEA multivariate archive spanning human motion, physiological signals, industrial sensors, and different channel counts [Bagnall et al., 2018]. The selection should be frozen before results are inspected. Candidate datasets include BasicMotions, CharacterTrajectories, Epilepsy, FingerMovements, Handwriting, MotorImagery, NATOPS, and SelfRegulationSCP1.

EMG.

- **NinaPro DB6**: cross-session and cross-day grasp recognition [Palermo et al., 2017].
- **EMGBench**: official intersubject and adaptation protocols across nine datasets [Yang et al., 2024].
- **NinaPro DB3**: leave-one-subject-out or held-out clinical population experiment with transradial amputees [Atzori et al., 2015].

6.3 Experiment E0: reproduction and adapter parity

Reproduce the frozen, head-only, full fine-tuning, LoRA, FourierFT, ChannelMixing, and Time-PEFT results on a manageable subset: Lorenz, ECGCA115, ETTh1, and Weather. Verify adapter placement, parameter counts, and head architecture before generating utility labels. TRACE and TS-PET reproductions may begin from official code or carefully isolated subcomponents if full code is unavailable.

6.4 Experiment E1: oracle adaptation map

Evaluate every MVP action on source-dataset episodes. The objectives are:

1. quantify how often the oracle action differs across datasets, horizons, and local regimes;
2. measure the regret of one globally fixed configuration;
3. determine whether Time-PEFT’s two complexity scalars alone predict the oracle action;
4. test whether the action ordering is stable across seeds and support sizes.

This experiment can falsify the project early. If one fixed action is nearly always optimal after cost matching, a learned selector is unnecessary.

6.5 Experiment E2: utility prediction on held-out forecasting datasets

Use leave-one-dataset-out meta-evaluation. For each held-out dataset:

1. train the utility predictor on all other datasets;
2. expose only the held-out support episode and action metadata;
3. select an action without target query labels;
4. adapt on support and evaluate on the official test/query partition;
5. compare with the target oracle, best source-fixed action, and target-specific search.

A stronger leave-one-family-out protocol holds out all chaotic, all medical, or all standard datasets.

6.6 Experiment E3: evidence ablation

Train identical predictors with:

1. Time-PEFT complexity scalars only;
2. structural descriptors without transfer entropy;
3. frozen-model mismatch only;
4. gradient probes only;
5. structure + mismatch;
6. structure + gradient;

7. mismatch + gradient;
8. all evidence.

This directly establishes whether the novelty comes from utility-oriented model evidence rather than reusing transfer entropy.

6.7 Experiment E4: cross-task transfer to classification

Meta-train the policy on forecasting only, then test zero-shot policy transfer to classification. Compare with a policy meta-trained on mixed forecasting and classification episodes. This separates general adaptation-value learning from forecasting-specific heuristics.

6.8 Experiment E5: NinaPro DB6 chronological adaptation

Use subject-disjoint and chronological splits. Two proposed tracks are:

Few-shot session adaptation. Train the backbone/controller without the target subject. Use a small labeled calibration subset from the first target session or day, select an action, and evaluate on later sessions chronologically. Report support fractions such as 1%, 5%, and 20% where feasible.

Rolling session adaptation. At session s , the controller uses only data from sessions $\leq s$ and predicts the action for session $s + 1$. Compare reset, persistent, and reselected adapters. No future session statistics may be used.

Key analysis: whether channel-heavy actions are selected after electrode re-donning, frequency-related actions under fatigue-like drift, and no-op/head-only actions when the frozen model remains adequate. These interpretations are hypotheses, not hard-coded rules.

6.9 Experiment E6: EMGBench and NinaPro DB3

Follow official EMGBench intersubject and adaptation splits, preserving its preprocessing and evaluation protocol [Yang et al., 2024]. Hold out entire EMG datasets from controller training where possible. NinaPro DB3 is a dedicated transfer test: compare a controller trained on non-EMG time series, on intact-subject EMG, and on other amputee subjects.

6.10 Experiment E7: extended rank and layer action space

After the MVP succeeds, expand G_a and r_a :

$$G_a \in \{\text{last block, last half, all blocks}\}, \quad r_a \in \{2, 4, 8, 16\}. \quad (17)$$

Compare direct action enumeration, factorized module/pair utility, AutoLoRA, AdaLoRA, dEBORA, FlexLoRA, TRACE, and TS-PET. This is the experiment most directly related to the professor’s questions about rank and layer selection.

7 Baselines and fair comparison

Table 2: Baseline families and their role.

Baseline	Selection mechanism	Purpose
Frozen / zero-shot	None	Establish adaptation need.
Head-only	Fixed minimal update	Strong low-cost reference.
Full fine-tuning	All parameters	High-cost performance reference.

Baseline	Selection mechanism	Purpose
Fixed LoRA	Fixed layers and rank	Generic PEFT baseline.
FourierFT	Learned sparse Fourier coefficients	Frequency-domain generic PEFT baseline [Gao et al., 2024b].
Time-PEFT	Fixed LoRA + frequency + channel adapters	Direct time-series-specialized baseline [Na et al., 2026].
AdaLoRA	SVD-based adaptive budget allocation	Global adaptive-rank comparator [Zhang et al., 2023].
AutoLoRA	Meta-learned rank-one selection	Closest meta-rank comparator [Zhang et al., 2024].
dEBORA / FlexLoRA	Bilevel or entropy-guided rank allocation	Modern adaptive-rank controls [Zangrando et al., 2025, Liu et al., 2026].
TRACE	Target-specific masking-based LoRA module importance	Reversible target-specific selection [Li and Zhu, 2026].
TS-PET	Interaction-aware pruned LoRA	Rank-interaction comparator [Zheng and Jin, 2026].
AT4TS	Target-specific HPO / selective tuning	Search-cost comparator [Tomar et al., 2025].
MixFT	Subdomain discovery + adapter library	Regime-routing comparator [Lee et al., 2026].
CoTo	Stochastic adapter activation + marginal contribution	Training robustness and utility-estimation comparator [Zhuang et al., 2025].
Complexity threshold router	Hand-coded Time-PEFT statistics	Tests whether learning utility is needed.
Random budget-matched action	Random feasible action	Detects action-space or regularization artifacts.
Best source-fixed action	One action selected on source validation	Primary fixed-policy baseline.
Target oracle	Exhaustive target query selection	Nondeployable upper reference and regret denominator.

Every comparison must match or explicitly report the same backbone and checkpoint, support size, label and validation access, optimizer and update steps, active-parameter budget, and measured compute budget. Methods using a target validation set, such as HPO or target-specific importance selection, should be reported in a separate information-access tier from amortized zero-search selection.

8 Metrics and statistical analysis

8.1 Task metrics

Forecasting uses MSE, MAE, and optionally MASE or sMAPE when scale comparability matters. Classification uses accuracy, macro-F1, balanced accuracy, and expected calibration error. Results are reported per dataset, subject/session, and task family rather than only as a grand average.

8.2 Selection metrics

Oracle regret.

$$R_{\text{oracle}}(e) = U(e, a_e^*) - U(e, \hat{a}_e). \quad (18)$$

Ranking quality. Spearman correlation and NDCG between predicted and observed action utilities; top-1 and top- k action accuracy.

Negative adaptation rate.

$$\text{NAR} = \frac{1}{N} \sum_e \mathbb{I}[\ell_{\hat{a}_e}(e) > \ell_0(e)]. \quad (19)$$

Efficiency. Active trainable parameters, stored adapter parameters, backward FLOPs, peak GPU memory, selection overhead, and adaptation wall time.

Pareto analysis. Compare methods on performance versus active parameters, performance versus backward FLOPs, and performance versus wall time.

8.3 Uncertainty and tests

Use at least three adaptation seeds for exploratory experiments and five for final key tables. For multiple datasets, report paired bootstrap confidence intervals and Wilcoxon signed-rank tests over dataset/horizon or subject/session units, with Holm correction. Effect sizes should accompany p -values. Utility-predictor calibration is measured by reliability diagrams or interval coverage.

9 Ablations and falsification criteria

9.1 Essential ablations

1. Complexity-only versus utility-oriented evidence.
2. With and without transfer entropy; scalar TE versus lag-resolved graph summaries.
3. With and without frozen-model mismatch features.
4. With and without gradient probes.
5. Individual module scores versus pairwise interaction terms.
6. Exact utility labels versus low-fidelity/successive-halving labels.
7. Regression only versus ranking only versus combined objective.
8. Fixed action cost weights versus hard budgets and budget-conditioned controller.
9. Dataset-level, session-level, and local-window episodes.
10. Forecasting-only versus mixed-task meta-training.
11. MOMENT-only versus MOMENT-to-UniTS transfer.

9.2 Falsification criteria

The central hypothesis is weakened or rejected if:

1. one globally fixed action matches Utility-PEFT on held-out datasets after cost matching;
2. a two-threshold complexity router matches the learned utility predictor;
3. gradient or mismatch evidence does not improve action ranking beyond Time-PEFT statistics;
4. utility gains vanish when target-validation access and compute are matched;
5. the policy collapses to nearly the same action for all targets;
6. low oracle regret occurs only on datasets used to meta-train the controller;

7. improvements appear only on EMG and do not reproduce on non-biomedical tasks;
8. policy-inference overhead offsets the saved adaptation cost;
9. utility labels are too unstable across seeds to support reliable ranking.

10 Expected results and interpretation

The expected result is not universal dominance on every dataset. Static Time-PEFT may remain best on homogeneous high-complexity forecasting datasets, and head-only or LoRA may remain sufficient on standard datasets. The proposed method should be strongest where adaptation needs vary across datasets, horizons, tasks, subjects, or sessions.

A successful result pattern would show:

1. lower oracle regret than complexity thresholds and best source-fixed policies on held-out datasets;
2. comparable or better forecasting/classification performance with fewer active parameters than always-on Time-PEFT;
3. lower search cost than AT4TS, TRACE, or exhaustive target validation;
4. meaningful action variation rather than indiscriminate activation of all modules;
5. transfer from forecasting meta-training to classification and EMG;
6. reduced negative adaptation on NinaPro sessions where large updates are harmful.

The most important scientific finding would be that target structure alone is insufficient: adaptation value becomes predictable only when structural descriptors are combined with evidence about the frozen model and the local optimization geometry.

11 Risks and contingency plan

Combinatorial action cost. Begin with seven actions and exact labels. Expand only after utility ranking works. Use low-fidelity evaluation and factorization for larger spaces.

Utility labels are noisy. Average over seeds, use ranking losses, model uncertainty, and treat near-ties as equivalent rather than forcing arbitrary labels.

Transfer entropy is expensive or unstable. Treat it as optional. Compare lagged correlation, mutual information, PCA/grouped TE, and learned channel-graph summaries. The paper’s novelty does not depend on TE.

Task mismatch. Separate task-specific heads and normalize utilities. Evaluate cross-task transfer explicitly rather than assuming one policy works unchanged.

Backbone integration differences. Freeze the primary claim on MOMENT before extending to UniTS. Document exact adapter targets per architecture.

No benefit over fixed policies. The oracle-map experiment is an early stop. A negative result can still support a useful article if it shows that adaptation complexity does not justify a controller under realistic budgets, but the full implementation should not proceed blindly.

Table 3: Indicative execution plan. Duration depends on official-code availability and GPU throughput.

Milestone	Deliverable	Main compute
M1	Reproduction report and tested adapter interfaces	Single GPU, small dataset subset
M2	Exact MVP utility table and oracle-map analysis	Parallel action jobs on one or more GPUs
M3	Utility predictor and held-out forecasting results	Light controller training + cached adaptations
M4	UEA classification and cross-task transfer	Moderate GPU usage
M5	NinaPro DB6 / EMGBench experiments	Data preprocessing + moderate fine-tuning
M6	Extended rank/layer space and UniTS validation	Highest compute; run only after MVP success
M7	Source-free/online extension and final paper	Optional second-stage study

12 Resource plan and milestones

The expensive artifact is the utility table, not the controller. Action evaluations should be embarrassingly parallel and resume-safe. A first credible paper can be completed with MOMENT-base, the MVP action space, forecasting leave-one-dataset-out evaluation, one classification suite, and NinaPro DB6; the extended architecture matrix is not required for the initial contribution.

13 Relevance to assistive technology

Patient-specific assistive-device research demonstrates that generic designs can create misalignment or discomfort when individual biomechanics differ [Chen et al., 2024]. The same personalization principle applies to neural interfaces: a generic EMG decoder can degrade across users, sessions, electrode placements, and postures. NinaPro and EMGBench provide an open route to study this problem without making biomedical novelty the main claim. The proposed article remains a neural-network contribution, while upper-limb prosthetic control is a realistic and socially relevant validation domain.

14 Conclusion

Utility-PEFT reframes automatic time-series fine-tuning as prediction of the counterfactual value of adaptation. Rather than routing from transfer entropy or spectral entropy alone, the method asks what a specific adapter operation is expected to improve for the current target episode and whether the gain justifies its cost. A modular action bank, frozen-model mismatch, gradient probes, interaction-aware utility prediction, and cross-dataset amortization create a clear novelty boundary relative to Time-PEFT, TRACE, TS-PET, AT4TS, MixFT, and adaptive LoRA methods. The proposed staged implementation is deliberately falsifiable: the oracle adaptation map must first demonstrate that meaningful selection exists, and the learned policy must then approach that oracle on held-out datasets, tasks, and EMG shifts under matched information and compute.

A MVP action-space specification

Table 4: Recommended first action space. All actions use identical support data, optimizer family, early-stopping policy, and maximum epochs.

ID	Active modules	Purpose	Controller action?
A0	Frozen / no update	Detect when adaptation is unnecessary	Yes
A1	Head only	Minimal adaptation	Yes
A2	Head + LoRA($r = 8$)	Generic PEFT	Yes
A3	Head + LoRA + frequency	Temporal-specialized adaptation	Yes
A4	Head + LoRA + channel	Multichannel-specialized adaptation	Yes
A5	Head + LoRA + frequency + channel	Static Time-PEFT-style full bank	Yes
A6	Head + FourierFT	Alternative compact spectral PEFT	Yes
A7	Full fine-tuning	Upper performance reference	No

After the MVP, factorize A2–A5 by layer group and rank. Avoid adding optimizer, loss, and persistence decisions until the structural selector is validated.

B Detailed dataset protocols

B.1 Forecasting

Use official or Time-PEFT-compatible chronological train/validation/test partitions. Meta-training episodes are drawn only from the training partition of source datasets. The source validation partition is used for global hyperparameters. Held-out target query/test labels are never used to train the utility predictor or select the action.

For each episode, sample a chronological support block and a subsequent query block. Prevent overlapping raw timestamps across support and query where window overlap would leak future observations. Report horizon-specific results before averaging.

B.2 UEA classification

Use official train/test splits. Within each source train split, create support/query meta-episodes with stratification by class while preserving grouped entities when metadata exist. Held-out target test labels are used only for final evaluation.

B.3 NinaPro DB6

Use subject-disjoint outer splits. Preserve session and day identifiers. Standardize using statistics from source training subjects and, when permitted, target support data only. Avoid random window splits that place repetitions from the same trial/session in support and query. Report per-subject and per-session metrics.

B.4 EMGBench and NinaPro DB3

Follow EMGBench’s official preprocessing, intersubject split, and adaptation fractions. For DB3, document exclusions, repetitions, gesture subset, and intact/amputee transfer setting explicitly. Clinical claims should remain limited to benchmark performance, not user benefit or safety.

C Algorithmic pseudocode

C.1 Offline utility-table generation

1. Load frozen backbone checkpoint and source dataset split.
2. Sample an episode $(\mathcal{D}_e^{\text{sup}}, \mathcal{D}_e^{\text{qry}})$.
3. Compute structural and frozen-model descriptors.
4. Initialize the full zero-impact module bank and collect one-step gradient probes.
5. For every feasible action a :
 - (a) restore identical backbone/head initialization;
 - (b) activate only modules in a ;
 - (c) adapt on $\mathcal{D}_e^{\text{sup}}$;
 - (d) evaluate on $\mathcal{D}_e^{\text{qry}}$;
 - (e) measure cost and compute $U(e, a)$;
 - (f) write an immutable utility record.
6. Repeat over episodes, datasets, seeds, and support sizes.

C.2 Utility-predictor training

1. Split utility records by dataset family, never by random rows alone.
2. Fit feature normalization on meta-training datasets only.
3. Train action-conditioned utility mean/uncertainty with regression and pairwise ranking.
4. Select hyperparameters by held-out source datasets, not episodes from the same dataset.
5. Calibrate predicted uncertainty on a separate source validation set.

C.3 Held-out deployment

1. Receive unseen target support episode.
2. Compute descriptors and gradient probes without query labels.
3. Score feasible actions under the deployment budget.
4. Select the highest conservative utility action.
5. Adapt selected modules on support.
6. Evaluate on target query/test data.

D Glossary of acronyms

Acronym	Definition
PEFT	Parameter-Efficient Fine-Tuning.
TSM	Time-Series Foundation Model.
LoRA	Low-Rank Adaptation.
SVD	Singular Value Decomposition.
FFT / IFFT	Fast Fourier Transform / Inverse Fast Fourier Transform.
TE	Transfer Entropy.
MVP	Minimum Viable Prototype.
HPO	Hyperparameter Optimization.

Acronym	Definition
OOD	Out-of-Distribution.
TTT / TTA	Test-Time Training / Test-Time Adaptation.
MSE / MAE	Mean Squared Error / Mean Absolute Error.
NDCG	Normalized Discounted Cumulative Gain.
NAR	Negative Adaptation Rate.
sEMG / HD-sEMG	Surface Electromyography / High-Density Surface Electromyography.
UEA	University of East Anglia time-series classification archive.

E Mathematical notation

Symbol	Meaning
f_θ	Pretrained TSFM backbone with frozen parameters θ .
e	Adaptation episode.
$\mathcal{D}_e^{\text{sup}}$	Episode support set used for adaptation.
$\mathcal{D}_e^{\text{qry}}$	Episode query set used to evaluate adaptation consequence.
$\mathcal{M}$	Bank of available adaptation modules.
$\mathcal{A}$	Set of feasible adaptation actions.
a	One action specifying modules, layers, rank, and update budget.
H, L, F, C	Task head, LoRA, frequency adapter, and channel adapter.
$\ell_0(e)$	Query loss before candidate adaptation.
$\ell_a(e)$	Query loss after adapting action a .
$\Delta(e, a)$	Normalized performance gain.
$\mathcal{C}(a)$	Resource cost of action a .
$U(e, a)$	Counterfactual utility of action a on episode e .
a_e^*	Nondeployable oracle action.
$\hat{U}_\psi(e, a)$	Predicted utility with controller parameters ψ .
$\mathbf{z}_e$	Episode evidence vector.
g_m	Gradient probe for module group m .
R_{max}	Maximum stored LoRA rank in the nested bank.
$g_{\ell j}$	Gate for rank-one unit j in layer ℓ .
B	Hard resource budget.
$\lambda_p, \lambda_f, \lambda_m, \lambda_t$	Cost weights for parameters, FLOPs, memory, and time.
R_{oracle}	Utility regret relative to the target oracle action.

F Codex implementation handoff checklist

Before large experiments, the implementation agent should verify:

1. The project reproduces one MOMENT forecasting baseline end to end.
2. Every adapter has a unit test proving zero-impact initialization.
3. Action activation changes only the intended trainable parameter groups.
4. Episode manifests prove chronological and subject/session separation.
5. Utility records can be regenerated from a config and checkpoint hash.
6. The no-op and head-only actions are always present.
7. Cost measurements are collected on the same hardware and batch size.
8. A tiny synthetic run completes utility generation, predictor training, and held-out selection in CI.

9. Results tables are generated programmatically from immutable records.
10. No target query/test label is exposed to descriptors, probes, or action selection.

References

- Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024.
- Manfredo Atzori, Arjan Gijsberts, Claudio Castellini, Barbara Caputo, Anne-Gabrielle Mittaz Hager, Simone Elsig, Giorgio Giatsidis, Franco Bassetto, and Henning Müller. The ninapro database: A resource for semg naturally controlled robotic hand prosthetics. *Scientific Data*, 2:150053, 2015. doi: 10.1038/sdata.2015.53.
- Anthony Bagnall, Hoang Anh Dau, Jason Lines, Michael Flynn, James Large, Aaron Bostrom, Paul Southam, and Eamonn Keogh. The uea multivariate time series classification archive, 2018. *arXiv preprint arXiv:1811.00075*, 2018.
- Trapit Bansal, Salaheddin Alzubi, Tong Wang, Jay-Yoon Lee, and Andrew McCallum. Meta-adapters: Parameter efficient few-shot fine-tuning through meta-learning. In *Proceedings of the First International Conference on Automated Machine Learning*, volume 188 of *Proceedings of Machine Learning Research*, pages 19/1–18. PMLR, 2022.
- Jacob Block, Sundararajan Srinivasan, Liam Collins, Aryan Mokhtari, and Sanjay Shakkottai. Provable meta-learning with low-rank adaptations. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- Chih-Yu Chen, Julia Martins Frejat, Ravi Vaidyanathan, and Klaus Drechsler. Biomechanical joint design of patient-specific rehabilitative knee exoskeletons for misalignment correction. In *2024 10th IEEE RAS/EMBS International Conference for Biomedical Robotics and Biomechatronics*, pages 801–805, 2024. doi: 10.1109/BIOROB60516.2024.10719729.
- Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10148–10167. PMLR, 2024.
- Vijay Ekambaram, Arindam Jati, Pankaj Dayama, Sumanta Mukherjee, Nam Nguyen, Wesley Gifford, Chandra Reddy, and Jayant Kalagnanam. Tiny time mixers: Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Shanghua Gao, Teddy Koker, Owen Queen, Thomas Hartvigsen, Theodoros Tsiligkaridis, and Marinka Zitnik. UniTS: A unified multi-task time series model. In *Advances in Neural Information Processing Systems*, volume 37, 2024a.
- Ziqi Gao, Qichao Wang, Aochuan Chen, Zijing Liu, Bingzhe Wu, Liang Chen, and Jia Li. Parameter-efficient fine-tuning with discrete fourier transform. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 14884–14901. PMLR, 2024b.
- Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. MO-MENT: A family of open time-series foundation models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 16115–16152. PMLR, 2024.

- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Thomas L. Lee, Edoardo M. Ponti, and Amos Storkey. Adapting time series foundation models through data mixtures, 2026.
- Yuze Li and Wei Zhu. TRACE: Time series parameter efficient fine-tuning. *Neurocomputing*, 664: 132098, 2026. doi: 10.1016/j.neucom.2025.132098.
- Muqing Liu, Chongjie Si, and Yuheng Jia. FlexLoRA: Entropy-guided flexible low-rank adaptation. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=tqnkdbdYWWm>.
- Yong Liu, Guo Qin, Zhiyuan Shi, Zhi Chen, Caiyin Yang, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Sundial: A family of highly capable time series foundation models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 39295–39317. PMLR, 2025.
- Jihye Na, Patara Trirat, Chanyoung Park, and Jae-Gil Lee. Time-PEFT: Temporal and multichannel complexity-based fine-tuning for time-series foundation models. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*. PMLR, 2026.
- Francesca Palermo, Matteo Cognolato, Arjan Gijsberts, Henning Müller, Barbara Caputo, and Manfredo Atzori. Repeatability of grasp recognition for robotic hand prosthesis control based on semg data. In *IEEE International Conference on Rehabilitation Robotics*, 2017. doi: 10.1109/ICORR.2017.8009289.
- Shivani Tomar, Seshu Tirupathi, Radu Marinescu, Elizabeth M. Daly, and Ivana Dusparic. AT4TS: Autotune for time series foundation models. *Transactions on Machine Learning Research*, 2025. URL <https://openreview.net/forum?id=U54YyLn8MX>.
- Jehan Yang, Maxwell Soh, Vivianna Lieu, Douglas J. Weber, and Zackory Erickson. EMGBench: Benchmarking out-of-distribution generalization and adaptation for electromyography. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, volume 37, 2024.
- Emanuele Zangrando, Sara Venturini, Francesco Rinaldi, and Francesco Tudisco. dEBORA: Efficient bilevel optimization-based low-rank adaptation. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=5M0ic2RxQZ>.
- Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=lq62uWRJjiY>.
- Ruiyi Zhang, Rushi Qiang, Sai Ashish Somayajula, and Pengtao Xie. AutoLoRA: Automatically tuning matrix ranks in low-rank adaptation based on meta learning, 2024.
- Liyang Zheng and Baisuo Jin. TS-PET: A novel framework for fine-tuning pretrained time-series models. *Big Data*, 2026. doi: 10.1177/2167647X261439005.
- Hongyun Zhou, Xiangyu Lu, Wang Xu, Conghui Zhu, Tiejun Zhao, and Muyun Yang. LoRA-Drop: Efficient lora parameter pruning based on output evaluation. In *Proceedings of the 31st International Conference on Computational Linguistics*, 2025.

Zhan Zhuang, Xiequn Wang, Wei Li, Yulong Zhang, Qiushi Huang, Shuhao Chen, Xuehao Wang, Yanbin Wei, Yuhe Nie, Kede Ma, Yu Zhang, and Ying Wei. Come together, but not right now: A progressive strategy to boost low-rank adaptation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 80533–80550. PMLR, 2025. URL <https://proceedings.mlr.press/v267/zhuang25c.html>.